

CLEAN AIR

Plano de Implantação

Professor Jorge Luiz
Implantação de Sistemas — G93686
Senai Lauro de Freitas
2026

Repositório do projeto:

github.com/jhonxxz06/TrabalhoLotDs2025.2-Senai

Este documento descreve o processo completo de implantação do sistema Clean Air em serviços de hospedagem em nuvem. Partimos do pressuposto que aqueles que vierem a ler não possuem cadastro em nenhuma das aplicações, com exceção do GitHub. O fluxo de implantação segue a seguinte ordem:

- Etapa 1 — Banco de Dados (Neon)
- Etapa 2 — Backend (Render)
- Etapa 3 — Frontend (Vercel)
- Etapa 4 — CI/CD (GitHub Actions)

Etapa 1 — Banco de Dados (Neon)

A configuração do banco de dados no Neon é um pré-requisito para as etapas seguintes. Após concluída, você deverá ter em mãos a `DATABASE_URL` que será utilizada nas variáveis de ambiente do backend.

1.1 — Criar Conta no Neon

1. Acesse neon.tech e clique em "Sign up". Crie sua conta utilizando GitHub, autorizando o acesso do Neon à sua conta.

1.2 — Criar o Projeto

2. Preencha os campos de configuração do projeto conforme abaixo:
 - **Project name:** nome de sua preferência (ex.: clean-air)
 - **Postgres version:** 16
 - **Region:** selecione a região mais próxima da sua aplicação (recomendado: AWS South America East 1)
3. Clique em "Create project" para confirmar.
4. Seguidamente, clique em "Go to project".

1.3 — Obter a `DATABASE_URL`

5. No painel do projeto recém-criado, clique no botão "Connect".
6. Na janela que abrir, copie a connection string clicando no botão "Copy snippet" — ela será a sua `DATABASE_URL`.
7. Guarde-a com segurança, pois será utilizada como variável de ambiente no Render.

⚠ **Nota:** Exemplo de `DATABASE_URL`:

`postgresql://usuario:senha@ep-exemplo-123456.us-east-2.aws.neon.tech/neondb?sslmode=require`

1.4 — Inicialização das Tabelas

8. Não é necessária nenhuma ação manual para criar as tabelas. Ao iniciar pela primeira vez no Render, o backend executa automaticamente `initDatabase()` (`index.js`), que chama `createTables()` (`database.js`), criando todas as tabelas via `CREATE TABLE IF NOT EXISTS`.
9. Para que isso funcione, basta garantir que a `DATABASE_URL` esteja corretamente configurada no Render conforme a Etapa 2.

Etapa 2 — Backend (Render)

O backend da aplicação será hospedado no Render. Antes de iniciar, é necessário que o repositório esteja na sua conta do GitHub via Fork.

2.1 — Fazer Fork do Repositório

10. Acesse o repositório original no GitHub: github.com/jhonxxz06/TrabalholotDs2025.2-Senai
11. Clique no botão "Fork" no canto superior direito da página.
12. Preencha o campo "Repository name" com o nome de sua preferência.
13. Clique no botão "Create Fork" para confirmar.

2.2 — Criar Conta e Projeto no Render

14. Acesse o Render em: render.com
15. Clique em "Start for free" e crie sua conta utilizando login via GitHub (isso facilita o acesso ao repositório forkado), novamente autorizando acesso de terceiros à sua conta.
16. No campo "Create a new service", selecione a opção "Web Services".

2.3 — Configurar o Web Service

17. Na tela de seleção de fonte de código, escolha a opção "Git Provider".
18. Selecione o GitHub como provedor.
19. Marque a opção "Only select repositories" e selecione o repositório fork criado.
20. Clique em "Install" e, em seguida, selecione o repositório.
21. Preencha os campos de configuração conforme abaixo:
 - **Name:** nome de sua escolha
 - **Language:** NODE
 - **Branch:** main
 - **Root Directory:** backend
 - **Build Command:** npm install
 - **Start Command:** npm run dev
 - **Instance Type:** de acordo com a necessidade do projeto
22. Preencha as variáveis de ambiente conforme a tabela abaixo (deixe o Caps Lock ativo ao digitar os nomes):

NAME_OF_VARIABLE	value
DATABASE_URL	{ URL obtida no Neon — etapa 1.3, passo 6}

NAME_OF_VARIABLE	value
FRONTEND_ORIGIN	{ deixar em branco provisoriamente }
JWT_EXPIRES_IN	24h
JWT_SECRET	{ código alfanumérico de sua preferência }
MQTT_BROKER	mqtt://broker.hivemq.com:1883
PORT	3001
BASE_URL	{ link copiado do Render — etapa 2.3, passo 24 }
TELEGRAM_BOT_TOKEN	{ código alfanumérico de sua preferência }
TELEGRAM_WEBHOOK_SECRET	{ código alfanumérico de sua preferência }

⚠ **Nota:** Exemplo `JWT_SECRET`: `J9.eyJrZXkiOiJ2YWx1` (código alfanumérico gerado por você)

23. Pressione "Deploy Web Service" para iniciar o deploy.
24. Na aba "Events", copie o link exibido em roxo — ele termina com ".onrender.com".
Guarde este link para uso nas etapas seguintes.

Etapa 3 — Frontend (Vercel)

O frontend será hospedado na Vercel. Certifique-se de ter o link do backend (Render) em mãos antes de iniciar esta etapa.

3.1 — Criar Conta e Importar Projeto

25. Acesse vercel.com e crie uma conta utilizando sua conta do GitHub.
26. Na tela de new project, vá em "Import Git Repository" > clique na opção do github
27. Selecione o repositório fork.

3.2 — Configurar o Projeto

28. Após selecionar repositório Configure o projeto preenchendo esses seguintes pontos:
 - Project Name: { preferência }
 - Application Preset: Create React App
 - Root Directory: clicar em "Edit" e selecionar Frontend > teste-mcp

3.3 — Configurar Comandos de Inicialização

29. Pressione "Build and Output Settings" e preenchendo os seguintes campos:
 - Build Command: deixa ativo e insere "npm run build"
 - Output Directory: deixa ativo e insere "build"

3.4 — Configurar Variável de Ambiente

30. Pressione "Environment Variables" e preencha assim:

Key	Value
REACT_APP_API_URL	{ link copiado do Render — etapa 2.3, passo 24 }

31. Clique em "Deploy" e aguarde
32. Terminado o deploy, pressione "Continue to Dashboard"
33. Na tela de "Overview" copie o link que está abaixo de "Domains"

3.5 — Finalizar a Configuração no Render

34. Retorne ao Render e acesse as variáveis de ambiente do seu Web Service.
35. Preencha o campo FRONTEND_ORIGIN com o link do Domain copiado da Vercel, precedido de https://:

KEY	VALUE
FRONTEND_ORIGIN	https://seu-projeto.vercel.app

⚠ **Nota:** Exemplo: *https://clean-air.vercel.app*

Etapa 4 — CI/CD (GitHub Actions)

Esta etapa configura os pipelines de integração e entrega contínua. Certifique-se de ter em mãos todos os valores das variáveis de ambiente das etapas anteriores antes de iniciar.

4.1 — Desativar o Auto-deploy no Render

36. Ainda no Render, acesse o campo "Settings".
37. No campo "Deploy", desative a opção "Auto-deploy" e salve a mudança.
38. No mesmo campo, copie o "Deploy Hook" e guarde-o para uso na etapa 4.2.

4.2 — Adicionar Secrets no GitHub

39. Acesse o repositório forkado no GitHub e clique em "Settings" (próximo ao topo da tela).
40. Acesse "Secrets and variables" > "Actions".
41. Na aba "Secrets", clique em "New repository secret" e adicione cada item da tabela abaixo. A cada novo secret, pressione "Add secret" antes de prosseguir para o próximo.

KEY	VALUE
DATABASE_URL	{ URL obtida no Neon — etapa 1.3, passo 6 }
FRONTEND_ORIGIN	{ link do Domain da Vercel — etapa 3.5, passo 35 }
JWT_EXPIRES_IN	24h
JWT_SECRET	{ o mesmo código utilizado no Render — etapa 2.3, passo 22 }
MQTT_BROKER	mqtt://broker.hivemq.com:1883
PORT	3001
REACT_APP_API_URL	{ link do backend no Render — etapa 2.3, passo 24 }
RENDER_DEPLOY_HOOK	{ Deploy Hook copiado do Render — etapa 4.1, passo 40 }

 **Nota:** Os nomes dos secrets devem ser escritos exatamente como na coluna KEY (com Caps Lock ativo). O campo Name corresponde à coluna KEY e o campo Secret corresponde à coluna VALUE.

4.3 — Habilitar e Executar os Workflows

42. Acesse o campo "Actions" próximo ao topo da tela do repositório.
43. Marque a opção "I understand my workflows, go ahead and enable them".
44. Selecione o workflow "CI Clean Air".
45. Pressione o botão "Run workflow" nas duas vezes que aparecer.
46. Aguarde a execução ser concluída com sucesso antes de prosseguir.

4.4 — Configurar Branch Protection Rule

47. Retorne a "Settings" novamente e acesse o campo "Branches".
 48. Clique em "Add branch ruleset".
 49. Defina o nome da ruleset (ex.: main-protection).
 50. Em "Enforcement status", selecione "Active".
 51. No campo "Branch targeting criteria", clique em "Add target", selecione "Include by pattern", digite main e confirme.
 52. No campo "Branch rules", marque as seguintes opções:
 - Restrict deletions
 - Require a pull request before merging e preencha:
 - Required approvals: 1
 - Dismiss stale pull request approvals when new commits are pushed
 - Require approval of the most recent reviewable push
 - Require conversation resolution before merging
 - Require status checks to pass e preencha:
 - Require branches to be up to date before merging
 - Em "Status checks that are required", clique em "Add checks", digite CI - Backend e selecione a opção do GitHub Actions
 - Repita o passo acima para CI - Frontend
 - Block force pushes
 53. Após marcar todas as opções, clique em "Create" para salvar a ruleset.
 54. Todas as etapas da Implantação concluída
-

Implantação Concluída

Finalizada a etapa 4.4, a implantação do sistema Clean Air está completa. O frontend estará acessível pelo link do Domain da Vercel, a comunicação com o backend estará configurada, o banco de dados em operação, assim como os scripts de CI/CD no GitHub Actions.